Technical Blueprint: Cyber Threat Intel

Github link: https://github.com/pranavsambidi/CyberThreatIntel

a. Problem Context and Project Summary

Modern Security Operations Center (SOC) environments operate under immense pressure. Tier-3 analysts and Threat Hunters are tasked with tracking Advanced Persistent Threats (APTs), but they face a critical bottleneck: intelligence consumption. Threat intelligence vendors (like Mandiant, Microsoft, and CISA) frequently publish high-fidelity incident reports, but these are almost exclusively delivered as unstructured, human-readable PDF documents.

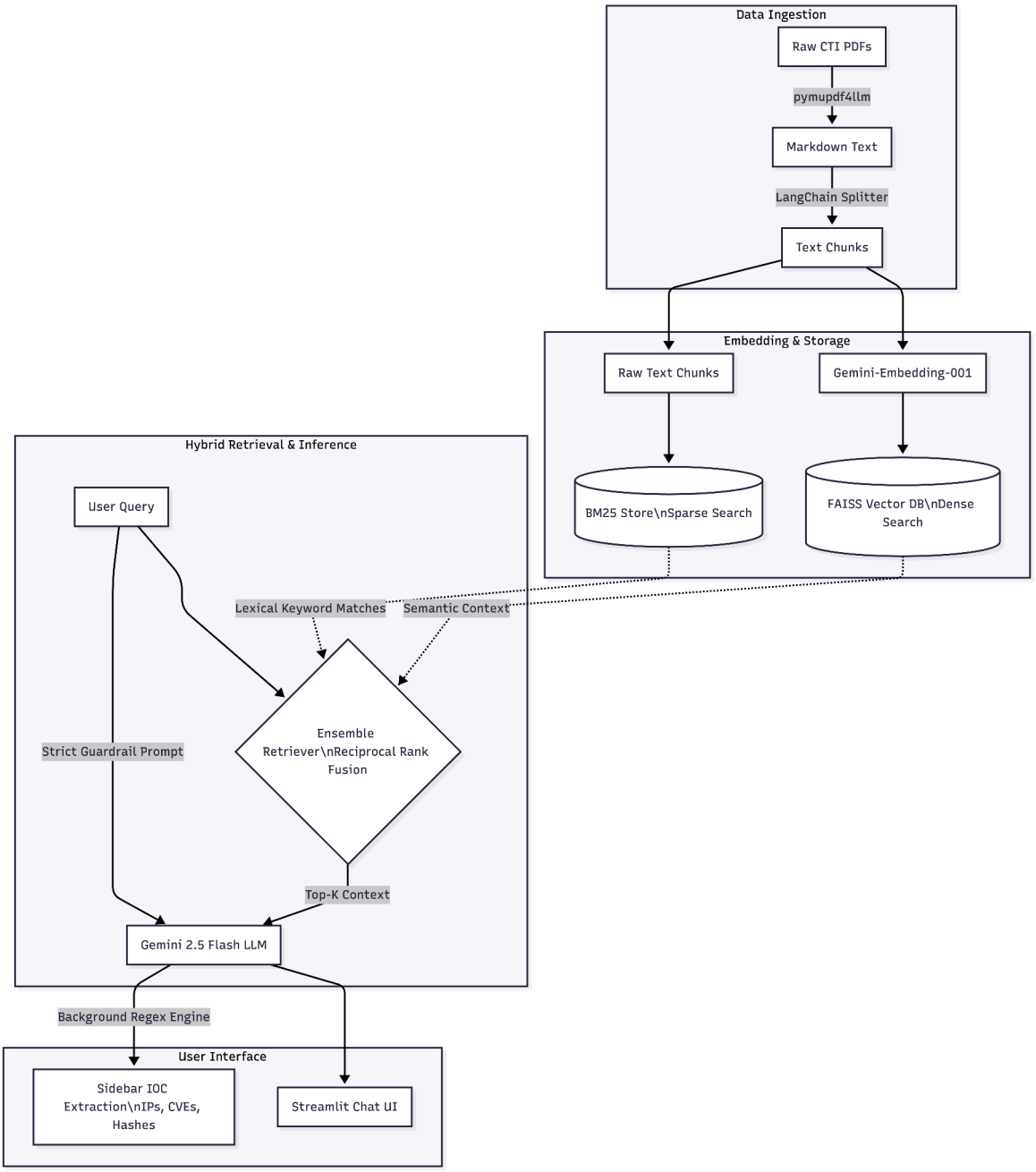
Manually reading a 40-page report to extract behavioral tactics, techniques, and procedures (TTPs), alongside exact Indicators of Compromise (IOCs), can take hours. This manual parsing drastically slows down incident response times and firewall patching.

My project solves this unstructured data problem by building an AI-powered Threat Intelligence Dashboard using a sophisticated Hybrid-Search Retrieval-Augmented Generation (RAG) architecture. This system ingests dense PDFs, preserves their tabular data, and creates a localized vector space. It allows analysts to query a private database of threat reports using natural language (e.g., *"What lateral movement tools does Volt Typhoon use?"*). Simultaneously, an automated Regular Expression (Regex) engine actively monitors the AI's output to isolate actionable IPs, Hashes, and CVEs, routing them to a clean interface for immediate network defense deployment.

b. Dataset

- **Source:** Public OSINT (Open-Source Intelligence) threat reports, primarily sourced from the CyberMonitor GitHub repository, as well as direct publications from cybersecurity research firms.
- **Type:** Unstructured PDF documents. These files contain dense technical narratives, network flow diagrams, and critical markdown-style tables containing IOCs.
- **Size:** The prototype currently operates on 6 foundational reports (approx. 166 vector chunks). For the final build, this will scale to an enterprise-level stress test of 30+ highly complex reports.
- **Engineering Challenges:** Standard PDF parsing libraries (like PyPDF2) are notorious for destroying tabular data layouts by reading tables line-by-line from left to right. Because malicious IPs and Hashes are almost exclusively stored in tables within these reports, standard parsing would corrupt the underlying intelligence.
- **Mitigation Strategy:** To preserve spatial relationships, the ingestion pipeline utilizes `pymupdf4llm` to extract the PDFs directly into raw Markdown format. This guarantees that tables are structurally preserved before they are passed to the chunking algorithm.
- **Ethical & Privacy Considerations:** Since this system utilizes the Google Gemini API for embeddings and inference, no proprietary or internal company network logs will be uploaded. The dataset is strictly confined to publicly available threat intelligence to ensure compliance with data privacy standards.

c. Planned Architecture



Data Flow:

The pipeline is divided into three distinct operational phases:

1. **Ingestion & Processing Phase:** Unstructured PDFs are converted to Markdown. The text is passed through LangChain's RecursiveCharacterTextSplitter (with strict overlap parameters to ensure context isn't lost mid-sentence). These chunks are vectorized using Google's gemini-embedding-001 model.
2. **Hybrid Retrieval Phase:** Relying solely on dense vector embeddings (Semantic Search) is a known failure point for alphanumeric string retrieval (like finding a specific SHA256 hash).

To counter this, the architecture uses a parallel retrieval method. It queries both a local FAISS Vector Store (for conceptual queries) and a BM25 Lexical Store (for exact keyword matching).

3. **Generation Phase:** The combined context is passed to the gemini-2.5-flash Large Language Model. The model is constrained by a strict System Prompt acting as a guardrail, before pushing the final narrative to the Streamlit user interface.

Algorithms, Frameworks, and Architectures Tested:

The core of this system relies on a Hybrid Retrieval-Augmented Generation (RAG) architecture. Traditional RAG relies exclusively on dense vector embeddings (like gemini-embedding-001 paired with a FAISS vector store) to perform semantic searches. While excellent for conceptual queries, dense search fails at exact alphanumeric string matching. To resolve this, a sparse lexical search algorithm (BM25) was integrated in parallel. The pipeline utilizes a LangChain Ensemble Retriever applying Reciprocal Rank Fusion (RRF) to merge the results of both algorithms. The retrieved context is then passed to a transformer-based Large Language Model (Gemini 2.5 Flash) for narrative synthesis and intelligence generation.

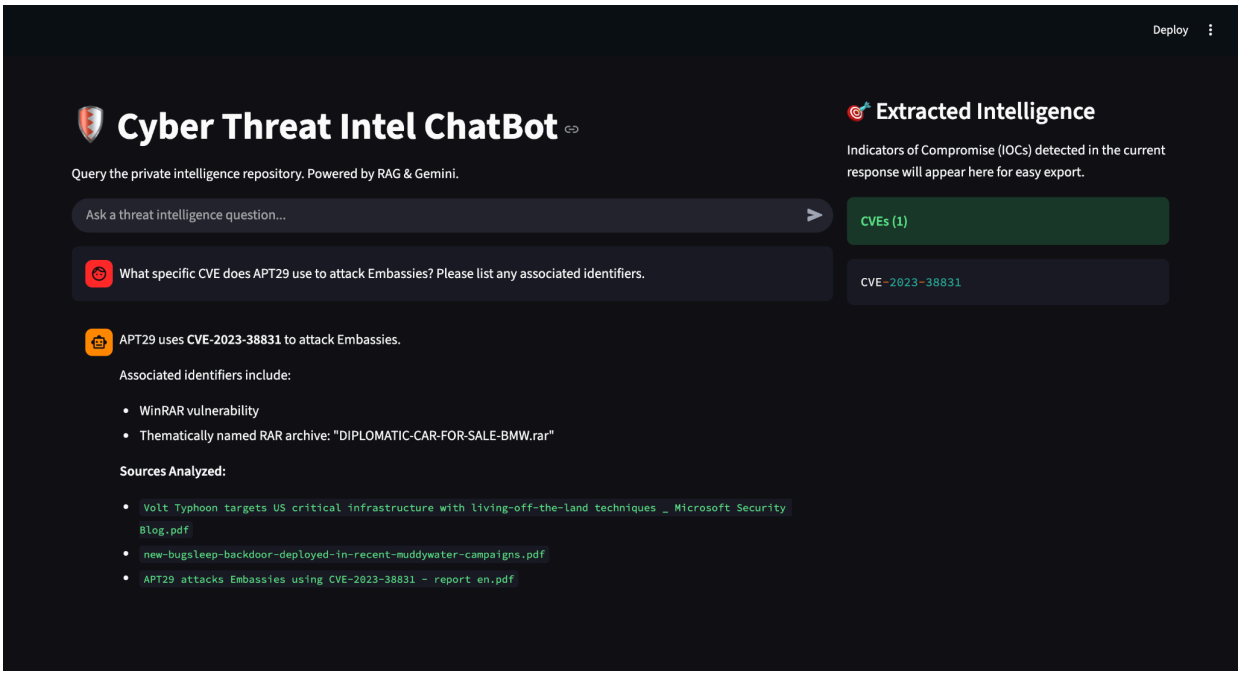
The Expected Role of the Interface:

The user interface is built using the Streamlit framework. Its primary role is to bridge the gap between complex vector mathematics and operational SOC workflows.

- **Input:** The UI provides a low-friction chat interface where analysts can submit natural language queries regarding the ingested threat landscape.
- **Outputs & Usability:** The interface enhances interpretability in two ways. First, it displays the LLM's analytical narrative alongside strict source citations to ensure data provenance. Second, it utilizes a background Regex engine that isolates actionable network artifacts (IP addresses, SHA256 hashes, and CVE identifiers) generated by the LLM and instantly routes them to a dedicated, copy-pasteable sidebar for immediate operational deployment.

d. User Interface Plan

- **Framework:** Streamlit (Python)
- **Design Persona:** The UI is designed specifically for Tier-3 SOC Analysts who require high information density and zero friction.
- **Input/Output Flow:** The user interacts via a standard chat interface. The system returns the generated intelligence, but critically, it appends explicit source document citations (e.g., Source: Microsoft_Volt_Typhoon_2024.pdf) at the bottom of every response. This provides the analyst with an immediate audit trail to verify the AI's claims.
- **Unique Feature (Automated IOC Extraction):** The interface features a dynamic 30% right-hand column split. As the LLM generates its response in the main chat, a background Regex engine scans the output stream. Any identified hard indicators (CVE identifiers, IP addresses, or SHA256 Hashes) are dynamically rendered in the sidebar as isolated, copy-pasteable code blocks, eliminating the need for the analyst to manually highlight and copy text from paragraphs.



e. Innovation and Anticipated Challenges

- **Key Innovation (Reciprocal Rank Fusion):** The implementation of an Ensemble Retriever combining FAISS and BM25 elevates this beyond a standard tutorial project. By fusing Dense and Sparse retrieval algorithms, the system guarantees high-fidelity recall for both broad behavioral questions and hyper-specific forensic identifiers.
- **Technical Challenge 1 - API Rate Limiting:** Free-tier API keys frequently crash during bulk data ingestion due to "Resource Exhausted" errors.
    - *Mitigation:* A custom rate-limiter was engineered into the `ingest.py` script. It processes exact batches of 50 chunks, forcing the thread to sleep for 30 seconds between batches, ensuring massive document scaling without API failure.
- **Technical Challenge 2 - LLM Hallucinations in High-Stakes Environments:** In cybersecurity, an AI hallucinating an IP address is catastrophic.
    - *Mitigation:* The LLM's temperature is set to `0.1` for maximum determinism. Furthermore, a strict parametric confinement prompt forces the model to reply verbatim with *"Insufficient data in current intelligence library"* if the answer cannot be mathematically tied to the retrieved vector chunks.

f. Implementation Timeline

- **April 1 – 5:** Establish virtual environment, construct data ingestion pipeline, implement Markdown extraction, and initialize FAISS vector database. *(Completed)*
- **April 5 – 12:** Develop Streamlit UI prototype, integrate the Regex sidebar engine, and successfully deploy the Hybrid Search (BM25 + FAISS) ensemble logic. *(Completed)*
- **April 12 – 19 (Current Phase):** Data Scale-Up. Ingest an additional 30+ enterprise threat reports to stress-test the local vector store and evaluate retrieval latency under heavy loads.

- **April 19 – 26:** Mathematical Evaluation. Write a programmatic testing script utilizing the RAGAS (Retrieval Augmented Generation Assessment) framework to quantitatively score the pipeline on "Answer Faithfulness" and "Context Relevance."
- **April 26 – 29:** Final dashboard refinement, generation of project documentation, and presentation preparation.

g. Responsible AI Reflection

The deployment of Generative AI within cybersecurity defense requires extreme caution. The primary risk of this system is the potential generation of "phantom indicators", instances where the LLM creates a syntactically correct but fundamentally false IP address or domain. If an analyst trusts this output blindly, they could inadvertently block legitimate network traffic, causing self-inflicted denial-of-service (DoS) on critical infrastructure, such as hospital databases or emergency response networks.

To practice Responsible AI, this project strictly adheres to a "Human-in-the-Loop" (HITL) philosophy. The AI is positioned as a research accelerator, not an autonomous decision-maker. By forcing strict source citations for every claim and implementing rigid prompt guardrails over creative generation, the system maintains transparency and defers the ultimate defensive action to the human analyst.